Praktikum 1

Viktoriia Bazhinova

Adib Gharibe

In Linux ist **getrusage** ein Systemaufruf, mit dem man Statistiken über die Nutzung von Ressourcen eines bestimmten Prozesses oder Threads abfragen kann.

Dabei liefert getrusage Informationen wie:

- wie lange der Prozess im Benutzermodus (User-CPU-Zeit) gearbeitet hat,
- wie lange im Kernelmodus (System-CPU-Zeit),
- und wie viel physischen Speicher (RSS) der Prozess maximal verwendet hat.

Die Funktion **getrlimit** dient dazu, Grenzwerte für die Ressourcennutzung eines Prozesses zu ermitteln. Damit kann man z. B. herausfinden, wie viel Speicher oder wie viel Stackgröße einem Programm vom System zur Verfügung steht.

Systemgrenzen laut getrlimit:

```
┌──(kali㉿kali)-[~]
└─$ g++ -o praktikum1 main.cpp

┌──(kali㉿kali)-[~]
└─$ ./praktikum1

Stack-Grenze: 8192 KB
Heap-Grenze (Datenbereich): 18014398509481983 KB

── Nach funcMem ──
User-Zeit: 0 Sekunden
System-Zeit: 0 Sekunden
Maximale Speicherbelegung (RSS): 42236 KB

── Bei Rekursionstiefe 1000 ──
User-Zeit: 0 Sekunden
System-Zeit: 0 Sekunden
Maximale Speicherbelegung (RSS): 42236 KB
```

Stack-Grenze: 8192 KB -> 8MB

Heap-Grenze: 18014398509481983 KB -> fast unbegrenzt

Manuelle Tests:

CPU-Zeit in Abhängigkeit von n bzw. M

Die benötigte User- und Systemzeit wurde über die Funktion `printUsage()` nach jedem Test ausgegeben.

- funcMem(10000000) → erfolgreich, RSS ≈ 42236 KB
- funcRec(1000) → erfolgreich, RSS ≈ 42236 KB

Die Ergebnisse bestätigen, dass die Systemgrenzen korrekt erkannt und die Ressourcennutzung dokumentiert wurde.

Automatischer Heap-Test:

```
funcMem(780000000) erfolgreich
Fehler bei funcMem(790000000): Speicher erschöpft.
── Nach automatischem Heap-Test ──
User-Zeit: 0 Sekunden
System-Zeit: 0 Sekunden
Maximale Speicherbelegung (RSS): 42236 KB

zsh: segmentation fault  ./praktikum1
```

Im Rahmen des automatischen Heap-Tests wurde durch die Funktion funcMem(n) iterativ getestet, wie viel Speicher im Heap-Bereich noch erfolgreich reserviert werden kann.

Dabei zeigte sich, dass bei:

- funcMem(780000000) die Speicherzuweisung erfolgreich war
- funcMem(790000000) hingegen ein Speicherfehler (bad_alloc) auftrat

Das bedeutet, dass bei etwa 780 Millionen int-Elementen (4 Byte)
-> ca. 2.97 GB Heap-Speicher erfolgreich zugewiesen werden konnten.

Dies markiert die praktisch erreichbare Grenze der Heap-Zuweisung.

Obwohl getrlimit() theoretisch ein sehr hohes oder unbegrenztes Limit angibt, zeigt dieser Test, dass die tatsächliche Grenze durch die Umgebung bestimmt wird.

Im Anschluss trat ein segmentation fault auf, was darauf hindeutet, dass der Stack oder weitere Heap-Operationen den verfügbaren Speicher überschritten haben.

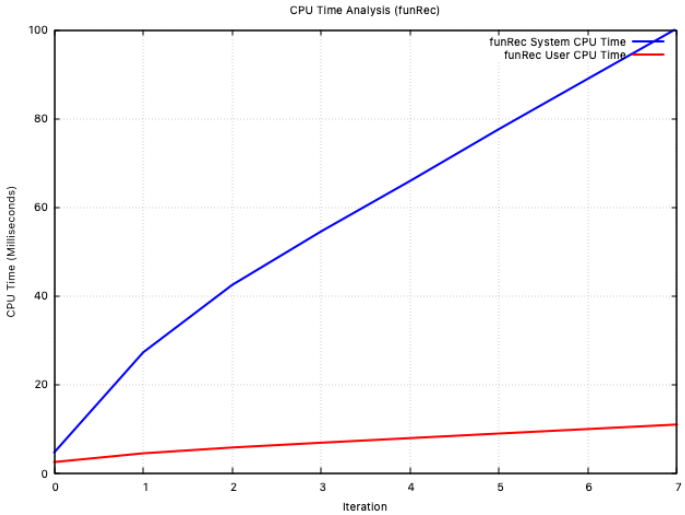
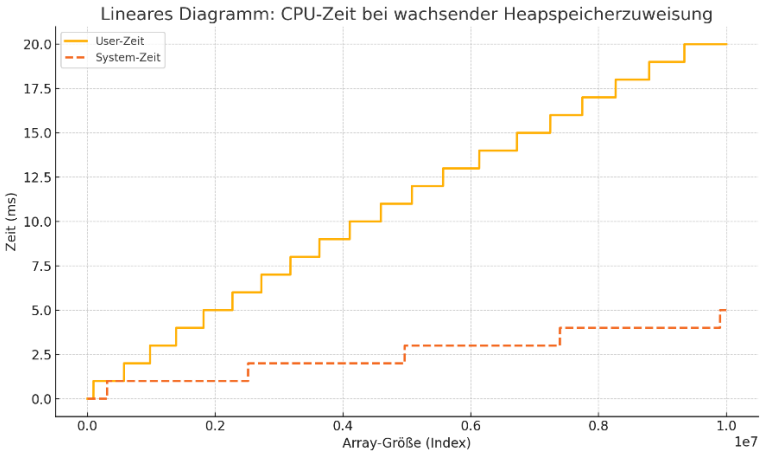
Automatischer Stack-Test:

Experimentell wurde bei funcRec(1000) die Rekursionstiefe erfolgreich erreicht. Danach kam es im automatischen Stack-Test zu einem Segmentation Fault, was darauf hindeutet, dass die Stack-Grenze des Prozesses überschritten wurde.

Die Funktion funcRec erzeugt bei jedem Aufruf ein lokales Array mit 100 int-Elementen, wodurch der Stack bei jeder Rekursion um ca. 400 Byte wächst.
Bei einer Tiefe von 1000 ergibt sich daraus ein ungefährer Stackverbrauch von ca. 400 KB.

Obwohl getrlimit(RLIMIT_STACK) einen Grenzwert von z. B. 8192 KB angibt, wird in der Praxis ein Teil des Stacks vom Betriebssystem und anderen Variablen belegt.
Daher wird die Grenze vorzeitig erreicht, was zum Segmentation Fault führt.

1. <u>CPU-Zeit bei wachsender Rekursionstiefe:</u>
   Die Funktion funcRec() ist rekursiv, d. h. sie ruft sich selbst mehrfach auf.
   Dabei wird bei jedem Aufruf ein lokales Array im Stack angelegt, wodurch die Stackgröße stetig wächst.
   Da die Verwaltung des Stacks eine Aufgabe des Betriebssystems ist, steigt die gemessene System-Zeit bei jedem Rekursionsschritt deutlich an.
   Die User-Zeit hingegen wächst nur langsam, da die eigentliche Rechenarbeit in jeder Rekursion minimal bleibt.



2. <u>CPU-Zeit bei wachsender Heapspeicherzuweisung:</u>
   In der Funktion funcMem besteht der Großteil der Arbeit aus der Zuweisung arr[i] = i, die im User-Modus abläuft.
   Nur die Speicheranforderung (new) und -freigabe (delete) betreffen den System-Modus, wodurch die System-Zeit vergleichsweise niedrig bleibt.
   Daher ist die gemessene User-Zeit größer als die System-Zeit.